

Fundamentos de Computação em Nuvem: Conceitos, Aplicações e Casos Reais

Curso: Análise e Desenvolvimento de Sistemas

Sumário

1. Para começar, uma pergunta	2
2. O que é computação em nuvem?	3
3. Onde a nuvem fisicamente existe?	4
4. Para que serve a nuvem? Por que as empresas migram?	8
5. Modelos de serviço: o que exatamente se "aluga"?	9
6. Modelos de implantação: onde a nuvem "mora"?	12
7. Onde a nuvem é utilizada? Setores e casos de uso	14
8. Estudos de caso: como empresas reais usam a nuvem, na prática	15
9. Quadro-resumo: empresa, provedor(es) e estratégia	17
10. Para refletir e discutir em sala	18

1. Para começar, uma pergunta

Antes de qualquer definição formal, pense na sua própria rotina digital de hoje:

Você abriu o Instagram, pediu comida pelo iFood, checkou o saldo no Nubank e assistiu a uma série na Netflix. Nenhum desses aplicativos guarda seus dados no seu celular. Onde, então, eles realmente "vivem"? Quem garante que o app do Nubank continue funcionando mesmo quando milhões de pessoas acessam ao mesmo tempo, no dia do pagamento do salário?

A resposta para essas perguntas é o assunto desta aula: **computação em nuvem**.

Nenhuma dessas empresas possui, fisicamente, todos os servidores que sustentam seus sistemas. Elas *alugam* poder computacional de terceiros — e é exatamente esse modelo que vamos entender.

2. O que é computação em nuvem?

Exemplo real antes da definição

O Nubank não comprou um único servidor físico para lançar seu primeiro cartão de crédito. Desde o primeiro dia, toda a infraestrutura do banco roda na AWS (Amazon Web Services) — o Nubank literalmente "**nasceu na nuvem**", como a própria equipe de engenharia descreve. Isso significa que, para escalar de alguns milhares para dezenas de milhões de clientes, a empresa não precisou construir um único data center: bastou solicitar mais capacidade computacional a um fornecedor.

Definição formal

Computação em nuvem (cloud computing) é o modelo de fornecimento de recursos de tecnologia da informação — servidores, armazenamento, banco de dados, redes, software — sob demanda, pela internet, **com pagamento conforme o uso**, sem que o cliente precise possuir ou administrar fisicamente essa infraestrutura.

O NIST (National Institute of Standards and Technology, dos EUA), referência clássica na área, define cinco características essenciais da nuvem:

Característica	O que significa
Autoatendimento sob demanda	O usuário provisiona recursos (ex: um servidor) sozinho, sem precisar de um atendente humano do provedor
Amplo acesso via rede	Os recursos são acessíveis pela internet, de qualquer lugar, por diferentes dispositivos
Pool de recursos	O provedor atende vários clientes com a mesma infraestrutura física, alocando recursos dinamicamente
Elasticidade rápida	A capacidade pode aumentar ou diminuir automaticamente conforme a demanda
Serviço mensurado	O uso é medido e cobrado de forma granular (por hora, por GB, por requisição)

3. Onde a nuvem fisicamente existe?

Uma pergunta que costuma surpreender

Se alguém perguntar "onde estão seus dados do Nubank?", a resposta **não** é "na nuvem", como se fosse um lugar abstrato, etéreo, sem endereço. Existe uma resposta muito mais concreta: existe um prédio, com paredes, ar-condicionado industrial, geradores a diesel, câmeras de segurança e milhares de servidores empilhados em racks — e esse prédio tem, literalmente, um endereço de rua.

Exemplo real: um dos principais data centers da AWS que atende o Brasil, chamado internamente de **GRU1**, fica na Avenida Ceci, 1900, no bairro Tamboré, em Barueri (Grande São Paulo). Esse único edifício tem capacidade elétrica estimada em cerca de **9 megawatts** — energia suficiente para abastecer uma pequena cidade — só para manter servidores ligados e refrigerados. E ele não está sozinho: a região de São Paulo da AWS é hoje composta por **5 data centers físicos distintos**, distribuídos entre Barueri, Jundiaí e Campinas.

Ou seja: quando um analista de sistemas do Nubank clica em "criar servidor" no painel da AWS, em algum lugar dentro de um desses prédios, um servidor físico real é reservado e configurado para atendê-lo — em poucos segundos.

"Nuvem" é, na verdade, uma rede de data centers reais

A "nuvem" nada mais é do que uma enorme rede de **data centers** (centros de processamento de dados) físicos, espalhados estrategicamente pelo mundo, conectados entre si por cabos de fibra óptica de altíssima velocidade — muitos deles, inclusive, cabos submarinos que atravessam oceanos inteiros.

Os grandes provedores organizam esses data centers em uma hierarquia geográfica:

Conceito	O que é	Exemplo
Região (Region)	Uma área geográfica ampla onde o provedor concentra infraestrutura	sa-east-1 — região "América do Sul (São Paulo)", ativa desde dezembro de 2011
Zona de disponibilidade (Availability Zone / AZ)	Um ou mais data centers fisicamente separados <i>dentro</i> de uma região, com energia, refrigeração e rede independentes entre si	A região de São Paulo da AWS possui 3 zonas de disponibilidade
Data center	O prédio físico em si: os racks de servidores, storage, rede, energia e refrigeração	O prédio GRU1, em Barueri
Edge location / PoP	Um ponto de presença menor, focado em entregar conteúdo com baixíssima latência para o usuário final (não roda toda a nuvem, só faz "cache" do conteúdo mais perto de você)	A AWS mantém pontos de presença em São Paulo e no Rio de Janeiro

Por que existem várias zonas de disponibilidade dentro da mesma região, e não um único data center gigante?

Porque isso é o que garante a **resiliência a falhas físicas**: se um incêndio, uma falha de energia ou uma inundação atingir o prédio de Barueri, o sistema de uma empresa como o Nubank pode continuar funcionando normalmente a partir de outro data center da mesma região, a poucos quilômetros de distância — sem o cliente sequer perceber.

Do hardware físico ao servidor que você "aluga": como a mágica acontece

Este é o ponto mais importante para tangibilizar o conceito: **como um prédio cheio de máquinas físicas se transforma no "servidor" que um desenvolvedor cria em segundos, digitando um comando?**

A resposta está em camadas de abstração, uma sobre a outra:

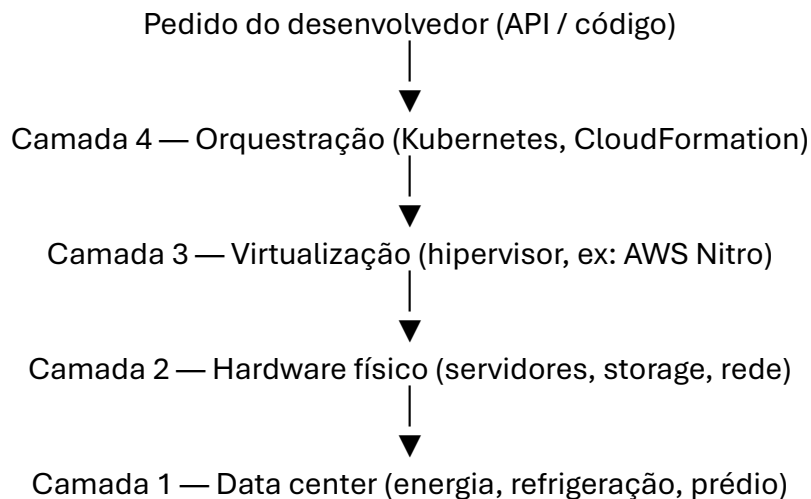
Camada 1 — Hardware físico: dentro do data center existem racks com centenas de servidores físicos reais (as mesmas peças que existem dentro de um computador comum, só que em escala industrial: processadores, memória RAM, discos SSD, placas de rede), interligados por switches de rede de altíssima velocidade, com sistemas redundantes de energia (incluindo geradores e baterias) e refrigeração.

Camada 2 — Virtualização (o hipervisor): um único servidor físico é potente demais para ser usado por um único cliente o tempo todo — seria um desperdício gigantesco de capacidade. Por isso, um software especial chamado **hipervisor** é instalado diretamente sobre o hardware físico e passa a "fatiar" aquele servidor real em dezenas de **servidores virtuais** independentes entre si, cada um isolado como se fosse uma máquina só sua, mesmo compartilhando o mesmo hardware.

Exemplo real: a AWS desenvolveu seu próprio hipervisor, chamado **Nitro**, especialmente desenhado para ser extremamente enxuto: em vez de o próprio processador principal gastar capacidade gerenciando rede e armazenamento (como hipervisores tradicionais faziam), a AWS move essas funções para chips físicos dedicados, colocados na placa-mãe ao lado do processador. O resultado prático é que o servidor virtual que você aluga (uma instância EC2) roda quase tão rápido quanto se você estivesse usando a máquina física inteira sozinho — e ainda com mais isolamento e segurança entre os clientes que dividem o mesmo hardware.

Camada 3 — Orquestração: com milhares de servidores virtuais espalhados por vários data centers, alguém — ou melhor, algum software — precisa decidir *onde* cada carga de trabalho vai rodar, reiniciar automaticamente o que falha, e distribuir tráfego entre réplicas. É aqui que entram orquestradores como o **Kubernetes**, coordenando mais de 30 mil microsserviços sem intervenção manual.

Camada 4 — API e painel de controle: por fim, tudo isso é exposto ao desenvolvedor através de uma interface simples — um comando de terminal, uma chamada de API, um clique em um painel web. Quando um engenheiro do Nubank sobe uma nova definição de infraestrutura via CloudFormation, ele não está mexendo em cabo nenhum: está enviando uma instrução que percorre essas quatro camadas até reservar, de fato, um pedaço de hardware físico dentro de um prédio em Barueri.



Para visualizar de verdade

Vale a pena, em sala, mostrar aos alunos imagens ou vídeos reais de data centers — a maioria dos grandes provedores disponibiliza tours virtuais e fotos oficiais de suas instalações (a Google, por exemplo, publica fotos de seus data centers com os corredores de servidores, tubulações de refrigeração e a famosa "sala dos cabos coloridos"). Ver fisicamente os racks de servidores ajuda a desfazer a ideia de que "a nuvem" é algo abstrato flutuando na internet — ela é, na verdade, um dos investimentos em infraestrutura física mais caros já construídos pela humanidade.

4. Para que serve a nuvem? Por que as empresas migram?

Exemplo real

Quando o iFood viveu um crescimento explosivo entre 2015 e 2016 — triplicando o volume de pedidos —, a infraestrutura própria da empresa simplesmente não aguentava. A solução não foi comprar mais servidores às pressas (processo lento e caro), mas migrar 100% para a AWS em 2016. Hoje, em horários de pico — como o almoço e o jantar de domingo —, a operação do iFood pode ultrapassar 500 instâncias de servidor ativas simultaneamente, processando dezenas de pedidos por segundo. Sem elasticidade de nuvem, esse tipo de pico sazonal exigiria comprar servidores que ficariam ociosos 90% do tempo.

Formalizando: os principais benefícios

- **Elasticidade e escalabilidade:** aumentar ou reduzir capacidade em minutos, acompanhando picos de demanda (Black Friday, horário de almoço, campanhas de marketing).
- **Redução de investimento inicial (CAPEX → OPEX):** em vez de comprar hardware (investimento de capital), a empresa paga uma despesa operacional recorrente, proporcional ao uso.
- **Agilidade e velocidade de inovação:** um time pode provisionar um ambiente de testes completo em minutos, em vez de esperar semanas por compra e instalação de hardware.
- **Foco no *core business*:** a equipe de TI deixa de gastar energia trocando disco de servidor ou atualizando firmware e passa a focar em construir produto.
- **Alcance global:** é possível colocar uma aplicação mais perto fisicamente do usuário final (ex: um servidor na região "São Paulo" da AWS) sem construir um data center no Brasil.
- **Resiliência e disponibilidade:** provedores de nuvem oferecem múltiplas zonas de disponibilidade e regiões, reduzindo o risco de uma falha física derrubar todo o sistema.

Esse conjunto de vantagens é resumido na frase que teremos como fio condutor desta unidade: "**You build it, you run it**" — **quem constrói o software também é responsável por operá-lo em produção**, algo que só se tornou viável em escala com a nuvem e as práticas de DevOps.

5. Modelos de serviço: o que exatamente se "aluga"?

Uma boa analogia para fixar: pense em pizza. **IaaS** é você alugar a cozinha e comprar os ingredientes (você cozinha). **PaaS** é receber a cozinha equipada e a massa pronta (você só monta e assa). **SaaS** é pedir a pizza pronta, entregue na porta (você só come).

Exemplo real

- Quando o time do Mercado Livre usa o **Amazon EC2** (máquinas virtuais) para rodar sua plataforma interna Fury, está consumindo **infraestrutura pura** — servidor virtual, rede, armazenamento — e precisa instalar e configurar tudo por cima.
- Quando uma equipe usa o **AWS Lambda** (como o próprio iFood usa) para rodar uma função sem se preocupar com o servidor por trás, está consumindo uma **plataforma** — o provedor cuida do sistema operacional, do runtime, do patch de segurança.
- Quando você usa o **Gmail** ou o **Slack**, está consumindo um **software pronto**, sem instalar nada, sem gerenciar servidor nenhum.

Formalizando: os três modelos clássicos

Modelo	O que o provedor entrega	O que você gerencia	Exemplos de mercado
IaaS (Infrastructure as a Service)	Servidores virtuais, redes, storage bruto	Sistema operacional, runtime, aplicação, dados	Amazon EC2, Google Compute Engine, Azure VMs
PaaS (Platform as a Service)	Ambiente de execução completo (runtime, middleware)	Apenas o código da aplicação e os dados	AWS Elastic Beanstalk, Heroku, Google App Engine
SaaS (Software as a Service)	Aplicação pronta para uso final	Apenas suas configurações e dados dentro do app	Gmail, Slack, Salesforce, Office 365

Como cada exemplo opera, na prática

IaaS — Amazon EC2, Google Compute Engine, Azure VMs

Nos três casos, o funcionamento é essencialmente o mesmo: o provedor pega um servidor físico do data center, usa um hipervisor (como o AWS Nitro3) para fatiá-lo em máquinas virtuais, e entrega a você uma dessas fatias como se fosse um computador vazio — com CPU, memória e disco reservados, mas **sem nenhum software instalado além do sistema operacional que você escolher**. A partir daí, toda a responsabilidade é sua: instalar o banco de dados, configurar o firewall, aplicar atualizações de segurança, escalar manualmente (ou programar a escala automática) quando o tráfego cresce.

Exemplo real: é exatamente esse modelo que o Mercado Livre usa como base da sua plataforma Fury — em vez de contratar servidores físicos próprios, a empresa provisiona instâncias EC2 (na AWS) e máquinas equivalentes no Google Compute Engine, e a própria Fury é o software que a empresa construiu por cima para automatizar tudo que, em um IaaS "cru", seria manual.

PaaS — AWS Elastic Beanstalk, Heroku, Google App Engine

Aqui o provedor entrega bem mais pronto: você envia apenas o código-fonte da sua aplicação (por exemplo, via git push), e a plataforma se encarrega de provisionar o servidor, instalar o runtime da linguagem (Java, Python, Node.js etc.), configurar o balanceador de carga e até escalar automaticamente o número de instâncias conforme o tráfego aumenta. Você nunca acessa o servidor diretamente — só entrega código e a plataforma decide onde e como rodar.

Exemplo real: no Heroku, cada aplicação roda dentro de um contêiner isolado chamado **dyno**; para escalar, o desenvolvedor simplesmente aumenta o número de dynos (ou o tamanho de cada um), sem nunca precisar configurar um servidor manualmente — foi esse modelo, lançado em 2007, que popularizou a ideia de "só dar git push e a aplicação sobe sozinha". Vale um adendo atual: em fevereiro de 2026 a Salesforce (dona do Heroku desde 2010) anunciou que o produto entrou em modo de "engenharia de sustentação" — seguirá recebendo atualizações de segurança e estabilidade, mas sem novos contratos empresariais nem grandes novidades, um lembrete de que até modelos de nuvem consolidados podem perder força de mercado com o tempo.

SaaS — Gmail, Slack, Salesforce, Office 365

Neste modelo não existe servidor, runtime ou infraestrutura visível em nenhum momento — o produto inteiro (a aplicação, o banco de dados, a interface) já está pronto e rodando na nuvem do fornecedor. O único trabalho do cliente é configurar o que a própria aplicação permite configurar (criar um usuário, montar um canal no Slack, definir permissões no Salesforce) e usar. Por trás da tela simples que você vê, o fornecedor de SaaS geralmente também está construído sobre uma base de IaaS ou PaaS de algum provedor — ou seja, os três modelos frequentemente se empilham.

Exemplo real: o Slack roda sua infraestrutura principal na AWS — ou seja, é uma empresa de SaaS que, por trás das cortinas, é cliente de IaaS da Amazon. Do ponto de vista de quem usa o Slack no trabalho, porém, nada disso importa: você só abre o navegador ou o aplicativo e conversa com o time, sem jamais pensar em servidor.

Uma observação importante: os modelos se empilham

Vale destacar em sala que essa divisão em três modelos não é uma escolha "ou um, ou outro" para uma empresa inteira — é comum que a mesma empresa combine os três ao mesmo tempo, dependendo da necessidade de cada parte do sistema. O Mercado Livre, por exemplo, usa IaaS (EC2/GCE) como base, constrói sua própria camada de PaaS interna (a Fury) por cima, e ainda contrata ferramentas de SaaS prontas (como Slack ou Salesforce) para processos internos que não fazem sentido desenvolver do zero.

6. Modelos de implantação: onde a nuvem "mora"?

Exemplo real

O Mercado Livre é um caso particularmente rico aqui: sua plataforma interna, chamada **Fury**, foi desenhada desde 2015 justamente para não depender de um único provedor. Hoje a empresa opera em **estratégia multi-cloud**, combinando AWS e Google Cloud Platform (GCP) por trás de uma camada de abstração (uma "camada anticorrupção") que impede o *lock-in* — ou seja, o time de desenvolvimento escreve código sem se prender à sintaxe específica de um provedor, e a empresa pode negociar custos ou trocar de fornecedor com muito mais liberdade.

Formalizando

Modelo	Descrição	Quando faz sentido
Nuvem pública	Infraestrutura compartilhada, oferecida por terceiros (AWS, Azure, GCP)	Startups, aplicações web, a maioria dos casos de uso atuais
Nuvem privada	Infraestrutura dedicada a uma única organização (própria ou hospedada)	Setores altamente regulados, dados extremamente sensíveis
Nuvem híbrida	Combinação de nuvem privada/on-premises com nuvem pública	Empresas em transição ou com requisitos específicos de compliance
Multi-cloud	Uso simultâneo de mais de um provedor de nuvem pública	Empresas que querem evitar dependência de um único fornecedor (<i>vendor lock-in</i>), como o Mercado Livre

Modelo	Descrição	Quando faz sentido	Exemplo real
Nuvem pública	Infraestrutura compartilhada, oferecida por terceiros (AWS, Azure, GCP)	Startups, aplicações web, a maioria dos casos de uso atuais	Netflix (AWS) e Spotify (Google Cloud) — rodam 100% na nuvem de terceiros
Nuvem privada	Infraestrutura dedicada a uma única organização (própria ou hospedada)	Setores altamente regulados, dados extremamente sensíveis	Nuvem de Governo (Serpro + Dataprev) — guarda dados sigilosos do governo federal brasileiro
Nuvem híbrida	Combinação de nuvem privada/on-premises com nuvem pública	Empresas em transição ou com requisitos específicos de compliance	Itaú Unibanco — ~65% já na AWS, restante ainda em infraestrutura própria/legada, com meta de 100% até 2028
Multi-cloud	Uso simultâneo de mais de um provedor de nuvem pública	Empresas que querem evitar dependência de um único fornecedor (vendedor lock-in)	Mercado Livre — plataforma Fury combina AWS e Google Cloud

7. Onde a nuvem é utilizada? Setores e casos de uso

A nuvem deixou de ser "uma opção de infraestrutura" para se tornar a espinha dorsal de praticamente todo setor da economia digital:

- **Fintechs e bancos digitais:** processamento de transações, prevenção a fraude, escalabilidade em datas de pico (ex: Nubank, PicPay, Inter).
 - **Marketplaces e e-commerce:** catálogo de produtos, recomendação personalizada, logística (ex: Mercado Livre, Amazon, Shopee).
 - **Delivery e mobilidade:** geolocalização em tempo real, matching entre pedido/entregador, picos sazonais previsíveis (ex: iFood, Uber, 99).
 - **Streaming de mídia:** distribuição de vídeo/áudio em larga escala, recomendação por machine learning (ex: Netflix, Spotify, Globoplay).
 - **Saúde:** prontuário eletrônico, telemedicina, análise de imagens médicas com IA.
 - **Educação:** plataformas de EAD, hospedagem de conteúdo, ambientes virtuais de aprendizagem.
 - **Governo:** portais de serviços públicos (ex: gov.br), que também usam infraestrutura de nuvem para lidar com picos de acesso.
 - **Inteligência artificial e dados:** treinamento de modelos de IA, que demandam grandes clusters de GPU disponíveis sob demanda — impraticável para a maioria das empresas manter fisicamente.
-

8. Estudos de caso: como empresas reais usam a nuvem, na prática

Esta seção aprofunda **exatamente como** cada empresa utiliza a nuvem — não apenas que elas "usam", mas quais serviços, arquiteturas e práticas adotam no dia a dia.

BR Nubank — infraestrutura imutável e *Infrastructure as Code*

O Nubank roda 100% na AWS desde a fundação. Um dos pilares técnicos mais interessantes da engenharia do Nubank é o conceito de **infraestrutura imutável**: em vez de alterar um servidor já em produção (o que gera inconsistências e dificulta rastrear problemas), toda a infraestrutura é descrita como código — em arquivos de definição versionados — e, a cada mudança, um conjunto *novo* de recursos é criado do zero, com o tráfego sendo redirecionado gradualmente do ambiente antigo para o novo (o chamado padrão **blue-green deployment**). Essa abordagem é orquestrada com o serviço **AWS CloudFormation**. Na prática, isso significa: nenhum engenheiro faz alterações manuais em um servidor "ao vivo" — tudo passa por código, revisão e automação, um dos pilares centrais da cultura DevOps que estudaremos nesta unidade.

BR iFood — de monólito a microsserviços orientados a eventos

O crescimento acelerado do iFood (triplicando o volume entre 2015-2016) forçou uma decisão arquitetural: sair de um sistema monolítico e migrar totalmente para a AWS em 2016, adotando uma **arquitetura de microsserviços**. Mais recentemente, a área financeira do iFood foi além, adotando **arquitetura orientada a eventos (Event-Driven Architecture)** para decompor um middleware financeiro monolítico — o resultado prático foi a redução do tempo de desenvolvimento de novas funcionalidades de **1 mês para 1 semana**. No dia a dia, o iFood combina múltiplos serviços gerenciados: **AWS Lambda** (execução de código sem gerenciar servidor), **EC2** (máquinas virtuais), **RDS** (banco de dados relacional gerenciado), **S3** (armazenamento de objetos), **CloudFront** (rede de distribuição de conteúdo) e **DynamoDB** (banco NoSQL), além de tecnologias como **PostgreSQL**, **Redis** (cache em memória) e streaming de eventos com **Apache Kafka** para rastrear pedidos e entregas em tempo real.

BR Mercado Livre — a plataforma interna Fury e a estratégia multi-cloud

Talvez o caso mais sofisticado entre os três: desde 2015 o Mercado Livre desenvolve o **Fury**, uma plataforma interna de desenvolvimento (*Internal Developer Platform*) construída sobre **Kubernetes**, que hoje orquestra mais de **30 mil microsserviços** e apoia cerca de **16 mil desenvolvedores**. O ponto central do Fury é abstrair a infraestrutura: os times de aplicação não escrevem código pensando diretamente em "AWS" ou "GCP" — existe uma camada de abstração que traduz essas chamadas para o provedor por trás, o que viabiliza a **estratégia multi-cloud** da empresa (combinando AWS e Google Cloud) sem *vendor lock-in*, além de dar ao time de infraestrutura liberdade para negociar custos ou migrar cargas de trabalho entre provedores conforme a necessidade.

Netflix — o pioneiro do "tudo na nuvem pública"

A Netflix é uma referência histórica: depois de uma corrupção de banco de dados em seu data center próprio em 2008, a empresa decidiu migrar totalmente para a AWS ao longo da década seguinte, tornando-se um dos maiores clientes de nuvem pública do mundo. A Netflix também é conhecida por criar e disponibilizar ferramentas open-source de resiliência, como o **Chaos Monkey** — um sistema que derruba servidores de produção *de propósito*, de forma aleatória, para forçar os times a construir aplicações que tolerem falhas naturalmente. É um exemplo direto de cultura DevOps aplicada à nuvem: assumir que a falha vai acontecer e projetar o sistema para se recuperar sozinho.

Spotify — nuvem para processamento de dados em larga escala

O Spotify migrou sua infraestrutura para o Google Cloud Platform, usando a nuvem tanto para servir música a centenas de milhões de usuários simultâneos quanto para o processamento massivo de dados que alimenta funcionalidades como o "Discover Weekly" — recomendações personalizadas geradas por pipelines de dados e machine learning que analisam o comportamento de escuta de cada usuário.

Amazon — quando o provedor de nuvem nasce da própria operação

Vale mencionar um caso curioso: a própria **AWS** nasceu da necessidade interna da Amazon.com de padronizar e escalar sua infraestrutura de e-commerce. Ao perceber que essa capacidade sobrando poderia ser vendida como serviço, a Amazon deu origem, em 2006, ao que hoje é o maior provedor de nuvem pública do mundo — mostrando que a fronteira entre "quem usa" e "quem oferece" nuvem pode até se confundir.

9. Quadro-resumo: empresa, provedor(es) e estratégia

Empresa	Provedor(es)	Estratégia principal
Nubank	AWS	Infraestrutura imutável + Infrastructure as Code (CloudFormation)
iFood	AWS	Microserviços orientados a eventos, serverless (Lambda)
Mercado Livre	AWS + GCP	Multi-cloud com plataforma interna (Fury) sobre Kubernetes
Netflix	AWS	Nuvem pública em escala global + engenharia de caos
Spotify	Google Cloud	Processamento de dados em larga escala para IA/recomendação
Amazon.com	AWS (própria)	Infraestrutura interna que se tornou o maior provedor do mercado

10. Para refletir e discutir em sala

1. Todos os casos brasileiros apresentados (Nubank, iFood, Mercado Livre) escolheram, em algum momento, migrar de uma arquitetura monolítica para microsserviços rodando na nuvem. Por que vocês acham que essas duas decisões — nuvem e microsserviços — costumam andar juntas?
 2. O Mercado Livre investe pesado em evitar *vendor lock-in* através do multi-cloud. Que desvantagens vocês imaginam que essa abordagem traz, em troca dessa liberdade?
 3. Pensando no papel do analista de ADS dentro de um time de produto: em qual desses exemplos você enxerga mais claramente a responsabilidade do **"you build it, you run it"**?
-

Fontes e leituras complementares

- [Estudo de caso da AWS: Nubank](#)
- [Infraestrutura imutável no Nubank — Blog da AWS](#)
- [Estudo de caso da AWS: iFood](#)
- [Arquitetura orientada a eventos no iFood — Blog da AWS](#)
- [Desenvolvimento de software para logística no iFood](#)
- [Kubernetes at Mercado Libre — Tecnología de Mercado Libre \(Medium\)](#)
- [The technological evolution at Mercado Libre: from the monolith to the multicloud platform](#)
- [Região da América do Sul da AWS \(São Paulo\)](#)
- [Regiões e Zonas de Disponibilidade da infraestrutura global AWS](#)
- [AWS South America Sao Paulo \(sa-east-1\) Data Centers](#)
- [What is a Hypervisor? — AWS Nitro System](#)
- [Salesforce puts Heroku out to PaaS — The Register \(fev/2026\)](#)

Nota: os exemplos internacionais (Netflix, Spotify, Amazon) refletem casos amplamente documentados e de conhecimento público no mercado de tecnologia; recomenda-se buscar os estudos de caso oficiais mais recentes de cada provedor de nuvem para atualização contínua deste material.